PROCESS SCHEDULING

OVERVIEW

You are to create a C++ program that simulates the 5 CPU scheduling algorithms covered in class. This involves examining the ready queue and repeatedly selecting one process for execution for a uniprocessor system.

GROUPING

To complete this project, you will work in groups of two (2) to three (3) members.

Instructors may, at their discretion, merge or reshuffle groups to ensure a manageable number of groups per section (typically 8–12, depending on class size).

SCORING

Having a working program will immediately net 60 points for your group. A total of 10 to 11 test cases will be provided. Minus 5 points for each simulation of a test case that is incorrect or lacking, up to -50.

For the defense, we will ask 2 major questions: explain a concept, edit your code, etc. The weight of each major question is 20 points. As an attempt to guide you to the answers, we may ask any number of minor questions when we are dissatisfied with your answers (the weight of each minor question is 0).

HPS is 100 points. Score cannot go below 0. Individual scores will be adjusted based on peer evaluations (mandatory by default, but your instructor might have made it optional).

INPUT FILE FORMAT

You are given an input file (all numbers are integers, no fractions):

- **First line:** A single number indicating the number of test cases.
- **Second line:** Start of first test case.

For each test case, at least 2 lines:

- **First line:** A number X greater than zero indicating number of processes, followed by a string: FCFS, SJF, SRTF, P, or RR.
  In the case of RR, another number Q greater than zero will follow.

- **Next X lines:** The X processes. Note that the first process' index is 1.
  Last process' index is therefore X and not X-1.

For each process, 1 line, 3 numbers:

- **First number:** Arrival time in ns, always zero or positive (note that test case start time is assumed to be 0ns but processes may not have arrived yet).

- **Second number:** Burst time in ns, always greater than zero.

- **Third number:** Nice level, range is -20 to +20, inclusive.

The string after X corresponds to the abbreviation of the algorithm to use:

- **FCFS** — First Come First Served
- **SJF** — Shortest Job First (non-preemptive)
- **SRTF** — Shortest Remaining Time First (SJF preemptive)
- **P** — Priority (preemptive)
- **RR Q** — Round-Robin (the number after the RR represents the time quantum).

For more details, refer to slide set on CPU scheduling. However, note that your simulator's round-robin behavior must mimic FCFS except that preempted processes are moved to the tail end of the queue and, if not currently running, must give way to new arrivals.

In the event of a tie (i.e. 2 or more processes are eligible to be chosen for CPU time), select the process with the earlier arrival time. If there is still a tie, select the process with the lower index.

SPECIFICATIONS

1. Assume a uniprocessor and zero overhead (context-switching is instant).

2. While a set of test cases will be provided on the day of your defense, it is assumed that you will create your own test cases for testing your simulation program.

3. For each test case, output a text version of the resulting Gantt chart:

    a. **First line**: Test case number (start with 1).
    b. **Second line**: First "block", corresponds to the first process to be run.

4. For each block:

    a. **First number**: Time elapsed so far in ns.
    b. **Second number**: Process index.
    c. **Third number:** CPU time used in ns. Add a capital X immediately at the end of this number if the process has completed at the end of this block.

5. Then, after the Gantt chart, you are to output the following:

    i. **<u>Total time elapsed</u>:** Remember that test case start time is always 0ns.

    Example:
    Last block in Gantt chart elapsed time 50ns + CPU time 30ns = 80ns

    ii. **<u>Total CPU burst time</u>:** This is not necessarily equal to total time elapsed since it is possible for the CPU to be idle while processes have not yet arrived.

    Example:
    i. Gantt chart
    1. Block 1 burst time: 25ns
    2. Block 2 burst time: 35ns
    ii. Total CPU burst time = 60ns

    iii. **<u>CPU utilization</u>:** To calculate this, you must determine the amount of time the CPU was not doing anything.

    Example:
    (100ns of total time - 0ns of idle) / 100ns of total time = 100%

    iv. **<u>Throughput</u>:** Number of processes completed per unit time.

Example:

4 processes / 100ns of total time = 0.04 processes/second

v. **Waiting time:** Total or sum of times waiting in the ready queue. For this project, display the 1) waiting time of each process; and 2) the average waiting time.

Example:

i. Waiting Time:

1. Process 1: 10ns

2. Process 2: 20ns

ii. Average waiting time = 15ns

vi. **Turnaround time:** Time it takes to execute the process, from submission (entry into the system) to completion (termination). For this project, output the turnaround time for each process.

Example:

i. Turnaround Time:

a. Process 1: 50ns

b. Process 2: 30ns

ii. Average turnaround time = 40ns

vii. **Response time:** Time it takes to start responding, from submission to first response.

Example:

i. Response Time:

1. Process 1: 0ns

2. Process 2: 20ns

ii. Average response time = 10ns

SAMPLE 1

INPUT:

```
2
4 SRTF
0 50 2
40 2 3
20 3 1
30 55 1
2 FCFS
100 10 1
10 70 1
```

OUTPUT (note the single whitespace before listing each individual process' waiting, turnaround and response times)**:**

```
1 SRTF
0 1 20
20 3 3X
23 1 17
40 2 2X
42 1 13X
55 4 55X
Total time elapsed: 110ns
Total CPU burst time: 110ns
CPU Utilization: 100%
Throughput: 0.0363636363636364 processes/ns
Waiting times:
 Process 1: 5ns
 Process 2: 0ns
 Process 3: 0ns
 Process 4: 25ns
Average waiting time: 7.5ns
Turnaround times:
 Process 1: 55ns
 Process 2: 2ns
```

```
 Process 3: 3ns
 Process 4: 80ns
Average turnaround time: 35ns
Response times:
 Process 1: 0ns
 Process 2: 0ns
 Process 3: 0ns
 Process 4: 25ns
Average response time: 6.25ns
2 FCFS
10 2 70X
100 1 10X
Total time elapsed: 110ns
Total CPU burst time: 80ns
CPU Utilization: 72%
Throughput: 0.0181818181818182 processes/ns
Waiting times:
 Process 1: 0ns
 Process 2: 0ns
Average waiting time: 0ns
Turnaround times:
 Process 1: 10ns
 Process 2: 70ns
Average turnaround time: 40ns
Response times:
 Process 1: 0ns
 Process 2: 0ns
Average response time: 0ns
```

SAMPLE 2

INPUT:

```
1
4 RR 25
0 30 3
25 45 4
75 10 1
55 15 5
```

OUTPUT (note the single whitespace before listing each individual process' waiting, turnaround and response times)**:**

```
1 RR
0 1 25
25 2 25
50 1 5X
55 4 15X
70 2 20X
90 3 10X
Total time elapsed: 100ns
Total CPU burst time: 100ns
CPU Utilization: 100%
Throughput: 0.04 processes/ns
Waiting times:
 Process 1: 25ns
 Process 2: 20ns
 Process 3: 15ns
 Process 4: 0ns
Average waiting time: 15ns
Turnaround times:
 Process 1: 55ns
 Process 2: 65ns
 Process 3: 25ns
 Process 4: 15ns
Average turnaround time: 40ns
```

```
Response times:
 Process 1: 0ns
 Process 2: 0ns
 Process 3: 15ns
 Process 4: 0ns
Average response time: 3.75ns
```

REQUIREMENTS (WHAT TO SUBMIT)

- Canvas submission must contain:
  - C++ code
  - Certificate of Authorship

- Group project defenses will be onsite.

- If you have questions, you may email your respective instructor. FAQ may be made available if there are a lot of common questions.

Good luck!